HOW MUCH HUMAN WORK IS BEHIND AI-ASSISTED SOFTWARE?
LABOUR BOUNDS AND ESTIMATION FROM WORK TELEMETRY

Abstract. Using artificial intelligence (AI) can change how events recorded by software development tools relate to human effort. Labour measures that effort in person-hours. To relate effort to delivery, we also define time to acceptance and an output score at follow-up. For each task, this score equals its initial reference score multiplied by the fraction of the accepted requirement still satisfied. Tasks missing the acceptance deadline receive an output score of zero. We examine what interval observations establish about labour and how sampled labour can correct a telemetry prediction of it.

For observed interval totals or justified bounds on them, each person's available time constrains the labour possible within a target window. Splitting time at the boundaries of observation intervals and target windows gives a finite linear programme with exactly the same possible labour totals, including when observations overlap or tasks share a person's time. Accounting jointly for shared capacity can rule out totals included when separately calculated interval bounds are added. Whenever the observations admit an allocation, the minimum, maximum and every total between them are attainable. Each total can also be realised with one activity at a time per person, provided activities may start and stop at arbitrary times.

For probability-sampled labour, we correct a telemetry prediction total with observed-minus-predicted differences weighted by inverse inclusion probabilities. Predictions are calculated before sampling and kept unchanged across possible samples. With known positive inclusion probabilities and accurately measured sampled labour, the corrected estimate averages to true labour over repeated samples. Its variance can be higher or lower than that of estimation without predictions. Analytical proofs, Lean-checked interval-integral and schedule-construction lemmas, and exact finite calculations establish the results within these allocation and sampling models. Empirical validation of the proposed measurements is the next step towards using them to compare AI-assisted work.

1. What the observations determine

Suppose a team records three person-hours of work during a four-hour interval. How much work occurred during its first two hours? If one person did all the work, that person can supply at most two person-hours in either half. The first half must therefore contain between one and two person-hours. The person could work one hour in the first half and two in the second, or two in the first and one in the second, so both totals are possible. The interval total thus determines a range of first-half labour totals. Commit times can leave a further ambiguity, since the same event sequence can arise from different amounts of human activity.

Labour is human time allocated to the included work, measured in person-hours. For software work assisted by artificial intelligence (AI), this includes unsuccessful work and post-acceptance repair as well as the work leading to acceptance. To interpret this effort alongside what the work delivers, we also define elapsed time to first acceptance and retained output. Retained output equals the task's initial reference score multiplied by the fraction of its accepted requirement still satisfied at follow-up, with zero credit for tasks missing the acceptance deadline.

When tasks share a person's time, their allocations across tasks and types of work must sum to at most one. This is the capacity constraint used in the opening example. When this constraint and the observations permit at least one allocation, our linear programme gives

Date: 2 November 2026.

Key words and phrases. software engineering, partial identification, labour bounds, activity sampling, model-assisted estimation, AI assistance.

the minimum and maximum consistent labour totals. Every total between that minimum and maximum is possible under the same constraints. A higher minimum or lower maximum therefore needs additional observations or justified restrictions on how people allocated their time.

When exact labour totals are unavailable, a probability sample of person-intervals (time intervals for individual people) can support estimation. Predictions calculated before sampling supply an initial total, corrected by the sampled differences between observed and predicted labour, each weighted by inverse inclusion probability. With known positive inclusion probabilities and accurately measured allocations, this estimator is unbiased over repeated samples. The correction improves precision when its sampling variance is lower than that of the labour estimate without predictions. Telemetry can therefore help estimate labour even when commit events leave its exact value unresolved.

These two uses of observations provide different guarantees. An identification interval contains exactly the labour totals consistent with specified observations and allocation constraints. Sampling uncertainty describes how an estimate varies with the selected sample. Both concern the measurement of labour. Estimating how AI changes labour or output additionally requires a causal study design.

Standard Coder illustrates the distinction between telemetry-derived activity and measured labour. Wright and Ziegler's approach [24] infers coding activity from commit timing and predicts the resulting labels from code-change features. Standard Coding Hours can serve as a common reference index, with calibration against independently observed labour required for interpreting it as person-hours. AI may alter both how human activity produces commits and how much work a code change represents. Forsgren et al. place activity and efficiency within a broader account of developer productivity [12], while Mozannar et al. examine the interaction time costs of AI-assisted programming [19].

Existing AI studies measure different outcomes in different populations. Peng et al. examine completion time among completers of a programming exercise [20]. Becker et al. study experienced maintainers in familiar repositories, comparing log completion times after adjustment for forecast times [4]. Cui et al. study weekly completed pull requests in organisational experiments [8], using allocation to study groups as an instrument for adoption in their June 2025 manuscript [7, Section 4.1 and Table 3]. The studies differ in their populations, AI tools, working practices and measured outcomes. Their estimates therefore answer different questions. A subsequent report from Model Evaluation and Threat Research describes selection and concurrent-agent measurement difficulties, further motivating separate labour and elapsed-time records [5]. The calculations here use no data from these studies.

Sections 2 and 3 explain why commits alone need not determine person-hours and define labour, acceptance time and retained output. Section 4 derives the minimum and maximum possible labour totals, and Section 5 uses sampled labour to correct fixed telemetry predictions. Section 6 sets out how to validate these measurements before comparing AI tools and working practices. The proofs apply standard optimisation and sampling results to the model's constraints on allocating people's time. Appendix A gives a shortest-path solution for one person's work of one type on one task.

2. Why event telemetry alone need not identify labour

To examine what commits reveal about human activity, let a minute-indexed hidden variable $Z_t \in \{0, 1\}$ indicate an activity state, and let $C_t \in \{0, 1\}$ record a commit event. A fitted event model specifies how activity changes from minute to minute and how commits depend on that activity. Interpreting $Z_t = 1$ as "coding" requires validation against independently measured labour. Within the model, different activity distributions can produce the same commit distribution, as the following construction shows.

Proposition 2.1 (Failure of unrestricted identification)**.** *Suppose the states Z_t are independent Bernoulli variables with parameter $\pi \in (0,1)$, and, independently across minutes and of the states, B_t are Bernoulli variables with parameter $c \in (0,1)$. Define $C_t = Z_t B_t$. The joint distribution of all commit observations depends on (π, c) only through πc. In particular, $(\pi, c) = (1/2, 1/5)$ and $(1/4, 2/5)$ give identical commit distributions but different expected active times.*

Proof. Each C_t is Bernoulli with parameter $\theta = \pi c$, and the C_t are independent. For every $n \geq 1$ and every binary string $(e_1, \ldots, e_n)$,

$$\mathbb{P}(C_1 = e_1, \ldots, C_n = e_n) = \theta^{\sum_t e_t}(1-\theta)^{n-\sum_t e_t}. \tag{1}$$

Both parameter pairs give $\theta = 1/10$. In contrast,

$$\mathbb{E}\left[\sum_{t=1}^{n} Z_t\right] = n\pi, \tag{2}$$

which is $n/2$ minutes in the first model and $n/4$ in the second. Thus identical commit distributions can correspond to different expected numbers of active minutes. □

This construction belongs to the independent-state subfamily of a two-state hidden Markov model, in which the next activity state has the same distribution for either current state. It establishes an ambiguity in the unrestricted model family. Generic-identifiability results for finite-state hidden Markov models concern parameters outside exceptional sets and permit hidden-state relabelling [2, Section 6.1]. Identifying the parameters therefore requires an explicit account of the restrictions or observations that distinguish the candidate models. Restrictions on the activity model, informative code-change features or independent activity observations may supply that distinction.

Identifying the activity model leaves a further measurement question. Interpreting its activity states as human labour requires validation against independently measured person-hours. Under AI assistance, commits can continue while human attention is elsewhere, so the mechanism that produces commits may itself change. Measured interval totals constrain how much labour occurred during a specified period. To use those measurements, we first define which work belongs to each task and the periods over which it is counted.

3. Task accounting with explicit boundaries

3.1. **Defining tasks and allocating human time.** To count a task consistently as its commits and pull requests change, use its recorded requirement as the reference. Begin tracking task i at time s_i, when its requested behaviour, conditions for inclusion in the study and acceptance criteria are recorded. Task splits, mergers and scope changes can then alter what completion requires. Their treatment in the records and analysis must be defined before observing outcomes, with the history of changes to requirements and acceptance criteria retained for that analysis.

A split task can retain its identifier while separate records describe its components. Specify whether accepted components count separately or through the completed parent, with each contribution counted once. This choice affects when output is recognised. Suppose one component is accepted before a deadline and another afterwards. Counting the components recognises the first by the deadline, while counting their parent waits for both. Regrouping can therefore change counted output even when the component reference scores, defined in the next subsection, add to the parent's score.

AI access defines part of the working conditions being compared. Its specification must identify the available tools and model versions, together with the tasks and autonomous actions permitted. Named reviewers and release requirements describe the human oversight governing the resulting work. These records describe what each group is allowed to do. Actual use needs a separate record alongside access and working practices, because difficult

tasks may prompt more tool use while people may decline an available tool. Comparing groups by use alone can therefore mix the effect of access with differences in task difficulty.

The next accounting question is how much human time each task receives. Let $a_{ijr}(t)$ be the fraction of person j's time spent on task i doing work of type r, such as implementation or review, at calendar time t. Each person's allocations across tasks and types of work sum to at most one. Integrating these fractions over time and summing across people and types of work gives cumulative task labour. With finite sets of tasks, people and types of work, and measurable allocation functions, labour through task age $u \geq 0$, measured from the start of tracking, is

$$a_{ijr}(t) \geq 0, \qquad \sum_{i,r} a_{ijr}(t) \leq 1, \qquad H_i(u) = \sum_{j,r} \int_{s_i}^{s_i+u} a_{ijr}(t)\, dt. \tag{3}$$

With time in hours, $H_i(u)$ has units of person-hours. Summing across people allows pair work to contribute two person-hours in one elapsed hour. The allocation constraint limits each person's combined contribution during that hour, including when several sessions run concurrently.

Several AI-assisted workflows can execute concurrently for the same person. The allocation functions record the human time spent prompting, supervising, reviewing and integrating their results. A person can switch between these activities while the workflows continue running. Attributing fractions of the person's time across tasks preserves the shared capacity constraint and prevents the same human time being counted in full against several workflows.

For a constructed example, suppose two workflows run throughout an hour. Their operator spends ten minutes prompting one and twenty minutes reviewing the other in separate periods, with no other human time devoted to the included work during that hour. The labour total is half a person-hour. Record one hour of execution for each workflow in the computing-resource account described below.

Applying this account requires deciding which activities and shared contributions belong to each task. Include all work from specification and implementation through review, testing and repair. The inclusion rule must state whether attended waiting belongs in the total. Shared review and platform work require a rule for attributing person-hours to the tasks in Equation (3). Work that the observations leave unattributed needs a separate record.

Unattended tool execution belongs in the computing-resource account. Reporting agent running time requires an explicit definition of an agent instance and the resource categories being counted. Report that time alongside token use and charges, keeping their units separate from person-hours. Combining these quantities requires an explicit economic conversion and changes the question from labour to cost.

Elapsed time to acceptance records how long a task takes, including periods within its tracking window when it is queued or running unattended. Let T_i be elapsed time from the start of tracking to first acceptance. Starting this clock at s_i includes queueing and any reasoning and specification between s_i and the first commit. If the task is permanently abandoned before acceptance, put $T_i = \infty$. This records abandonment as an outcome, while loss of observation is censoring. Acceptance and deployment may occur at different times and then require separate records.

3.2. **Scoring tasks and checking retained output.** Comparing output requires a common reference for what each task delivers. Assign each task a nonnegative reference score S_i^0, recording the scoring-rule version and the task information it uses. The score may count tasks or functional units, or use Standard Coding Hours as weights. Retained output has the same unit as this score, and productivity is retained output per person-hour. The superscript 0 denotes the common reference. Keep that reference consistent by retaining each task's score and using the same scoring rule and weights across comparison groups.

The choice of scoring input determines whether that reference depends on the AI tool being compared. For a causal comparison of fixed task requirements, prefer the requirements

recorded when tracking begins. AI can change the final code, so a retrospective code score depends on the resulting implementation. Such a score can still be reported with this dependence made explicit. A nonlinear scoring rule also allows splitting or squashing a change to alter its score. To keep the reference consistent, score the same recorded task information once, or use a requirement-scoring rule whose component scores add to its total. Count the delivered requirement once across failed attempts, retries and reverts.

A score expressed in person-hours needs calibration against independently measured labour. That calibration must identify whose labour is included and the types of work counted, together with the quality requirements and the event at which labour recording ends. Training a predictor on imputed activity states teaches it to predict those states. Interpreting its predictions as person-hours requires independent validation against labour measured on the specified basis.

For each task accepted by the deadline, multiply its initial reference score by the fraction of its accepted requirement still satisfied at follow-up. This product is its retained output score. A task missing the deadline contributes an output score of zero. Choose an acceptance deadline at task age $\tau > 0$ and a follow-up duration $\Delta \geq 0$ beginning at first acceptance. Specify the criteria for judging what fraction of an accepted requirement remains satisfied. For accepted tasks, let $R_i(\Delta) \in [0, 1]$ be that fraction at time $s_i + T_i + \Delta$, judged using these criteria. Retained reference output is

$$Q_i(\tau, \Delta) = \begin{cases} S_i^0 R_i(\Delta), & T_i \leq \tau, \\ 0, & T_i > \tau. \end{cases} \tag{4}$$

As a constructed example, suppose a task has an initial reference score of ten units and is accepted before the deadline. A retained fraction of 0.8 yields an output score of eight reference units. Calculating that fraction requires a rule for assessing partial requirement satisfaction and distinguishing intentional replacement from defect-driven loss. Surviving lines, successful tests and absence of recorded incidents each provide incomplete evidence of requirement satisfaction. The follow-up procedure must document which defects its checks can detect and measure agreement between reviewers.

The labour and output accounts now give a productivity ratio. For a specified finite set of tasks with positive total labour, define

$$P_H(\tau, \Delta) = \frac{\sum_i Q_i(\tau, \Delta)}{\sum_i H_i(\tau + \Delta)}. \tag{5}$$

The denominator counts labour through the common task age $\tau + \Delta$ for every task in that set. It therefore includes labour on rejected, unfinished and abandoned tasks, together with attributable work after acceptance. Within this common window, early acceptance leaves more time for repair than late acceptance. Ending repair accounting exactly Δ after each acceptance would instead make the end of labour recording depend on acceptance time and define a different denominator. Measuring the ratio defined here requires recording when labour observation ends and following every task for $\tau + \Delta$.

An individual task with zero human labour can still contribute retained output. The positivity requirement applies to the tasks' combined labour and determines when the productivity ratio is defined. Report retained output and total person-hours separately so the ratio can be interpreted alongside its numerator and denominator.

3.3. **The acceptance deadline and observation periods.** Use the acceptance deadline τ to decide which tasks qualify for the output calculation. For a task accepted in time, assess requirement fulfilment at task age $T_i + \Delta$, using first acceptance as the start of follow-up. Count labour through task age $\tau + \Delta$ for every task, including unfinished and abandoned tasks. A complete task history must therefore include the acceptance record, the follow-up assessment and all attributable repair work within the labour observation period.

Complete follow-up can still leave several labour allocations consistent with the records. A record *point-identifies* realised labour when at least one time allocation is consistent with the recorded observations and model constraints, and every such allocation gives the same total. Here the question concerns the labour in a recorded history. Proposition 2.1 instead asks whether the probability distribution of observed events uniquely determines a population parameter. The next section derives the smallest and largest labour totals consistent with interval observations.

4. Attainable labour ranges from interval observations

An interval total can leave open when people worked within it. We seek the labour totals possible within the target windows under all observations and the allocation constraint in (3). The *identified range* contains exactly the totals obtainable by allocations satisfying these conditions. This definition follows the general partial-identification perspective [23, Section 1]. We begin with one person's work in one interval, then account jointly for overlapping observations and shared time.

4.1. Using the person's capacity.

One person can contribute at most one person-hour in an elapsed hour. Knowing who supplied an interval total therefore limits how much of that labour could have occurred before or after a specified time. A total across several people must be compared with their combined available time.

Proposition 4.1 (Attainable contribution from one interval)**.** *Let $\ell < u$, $c \in [\ell, u]$ and $v \in [0, u-\ell]$. Among measurable allocations $a : [\ell, u] \to [0,1]$ with $\int_\ell^u a(t)\,dt = v$, the attainable values of $x = \int_\ell^c a(t)\,dt$ are exactly*

$$[\max\{0, v-(u-c)\},\ \min\{v, c-\ell\}]\,. \tag{6}$$

If $v < 0$ or $v > u-\ell$, no such allocation exists.

Proof. Nonnegativity and capacity on the two pieces imply

$$0 \le x \le c-\ell, \qquad 0 \le v-x \le u-c.$$

Together, these inequalities give (6). Conversely, fix any x in this interval. For $\ell < c < u$, define the allocation function by

$$a(t) = \begin{cases} x/(c-\ell), & \ell \le t < c, \\ (v-x)/(u-c), & c \le t \le u. \end{cases}$$

Both allocation rates lie in $[0,1]$ by the preceding inequalities, so the function satisfies the capacity constraint. Its integrals over the two subintervals are

$$\int_\ell^c a(t)\,dt = \frac{x}{c-\ell}(c-\ell) = x,$$
$$\int_c^u a(t)\,dt = \frac{v-x}{u-c}(u-c) = v-x.$$

Adding them gives total labour v. When $c = \ell$, the interval for x forces $x = 0$. When $c = u$, it forces $x = v$. In either case, use the constant allocation $a(t) = v/(u-\ell)$ on the complete interval, so the zero-length subinterval contributes zero without division by its length. Finally, integrating $0 \le a \le 1$ gives the necessary condition $0 \le v \le u-\ell$. □

For one person, 3/2 person-hours in a two-hour interval imply between 1/2 and 1 person-hours in its first hour. At most one person-hour can fall in the second hour, so at least half a person-hour must fall in the first. The first hour's own capacity supplies the upper bound. Capacity therefore narrows the range permitted by nonnegativity alone, $[0, 3/2]$, while leaving the first-hour total and timing partly open.

If $v = u-\ell$, the total uses all the person's available time, so the labour before any time within it equals the elapsed time from its start. The same accounting extends to an interior

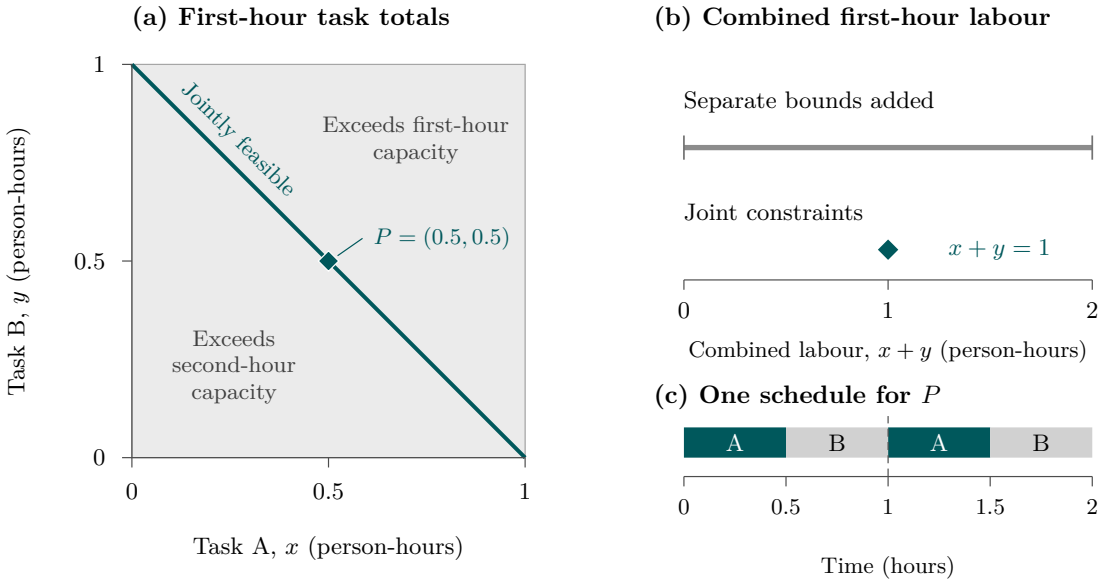


FIGURE 1. **Shared capacity determines the combined first-hour labour.** In this constructed example, one person spends one person-hour on each of two tasks during two hours. In (a), x and y are the tasks' first-hour labour totals. Every point in the shaded square satisfies the separate bounds. Enforcing the person's capacity in both hours restricts the feasible points to the diagonal. In (b), adding the separate bounds permits between zero and two person-hours, whereas the joint constraints require exactly one. In (c), one full-attention schedule attains the marked point $P = (0.5, 0.5)$ by giving each task half an hour in each hour, with one activity at a time. The example allows the person to switch tasks at arbitrary times. The schedule is neither unique nor observed.

subinterval $[b, c] \subseteq [\ell, u]$. Applying the construction to that subinterval and the time outside it gives every possible labour total in the subinterval, $[\max\{0, v-((u-\ell)-(c-b))\}, \min\{v, c-b\}]$.

Combining contributions requires distinguishing bounds on their sum from totals that can occur together. Adding separately calculated lower and upper bounds gives bounds on the combined total when each contribution is counted once. Attaining the summed lower bound requires all component minima together, and attaining the summed upper bound requires all component maxima together. Disjoint intervals of one person, or intervals belonging to distinct people, can be combined when no other constraints link them. Overlapping observations or several tasks competing for the same person's time require a joint calculation.

For example, suppose one person spends one person-hour on each of two tasks during $[0, 2]$. Each task separately allows $[0, 1]$ person-hours before time 1. Together, the tasks use all the person's available time on $[0, 2]$, so exactly one person-hour must fall before time 1. Adding the separately calculated bounds would instead give $[0, 2]$. Figure 1 shows how the shared capacity restricts the jointly feasible first-hour totals within the separately permitted ranges. Panel (c) gives one schedule for the marked midpoint, illustrating the reconstruction with one activity at a time proved below.

4.2. **Replacing allocation histories with interval totals.** The joint calculation must retain the capacity link between tasks sharing a person's time. Call one person's work of one type on one task a stream. Fix finite sets of tasks $\mathcal{I}$, people $\mathcal{J}$ and types of work $\mathcal{R}$, a set of tasks to be studied $\mathcal{C} \subseteq \mathcal{I}$, and a bounded calendar-time domain $[b_0, b_K]$ with $b_0 < b_K$

containing every observation interval and target window. An admissible history consists of measurable $a_{ijr} \geq 0$ satisfying $\sum_{i,r} a_{ijr}(t) \leq 1$ almost everywhere for every person j.

The list of people and the rules for attributing time to tasks determine whose labour and which work enter this calculation. Whether the records cover the intended work requires measurement evidence. Work outside $\mathcal{C}$ can be recorded separately when it uses the same people's time without contributing to the labour total for $\mathcal{C}$.

Each observation constrains the labour of specified streams during one interval. For observations $\nu = 1, \ldots, M$, specify an interval $[\ell_\nu, u_\nu]$ within this domain, a set $G_\nu \subseteq \mathcal{I} \times \mathcal{J} \times \mathcal{R}$ of streams whose labour the observation includes, and finite labour bounds $v_\nu^- \leq v_\nu^+$, in person-hours. These bounds impose

$$v_\nu^- \leq \sum_{(i,j,r) \in G_\nu} \int_{\ell_\nu}^{u_\nu} a_{ijr}(t)\, dt \leq v_\nu^+. \tag{7}$$

Exact interval totals use $v_\nu^- = v_\nu^+$. Justified limits on rounding or measurement error may instead give different lower and upper bounds. A Horvitz–Thompson estimate varies with the selected sample. Its sampling uncertainty must be accounted for before the estimate can justify either inequality.

Form the sorted distinct endpoints $b_0 < \cdots < b_K$ from the domain boundaries, every observation endpoint and every target boundary, and call the intervals between successive endpoints cells. Every observation interval and target window is then a union of complete cells, allowing its labour to be written as a sum of cell totals. For the denominator in (5), target boundaries include the start of tracking s_i and the end of the labour observation window $c_i = s_i + \tau + \Delta$ for every task in $\mathcal{C}$.

The partition uses calendar time so that simultaneous work by the same person shares one capacity constraint. Translate time since tracking began into calendar time before partitioning. Cells outside a target window must remain whenever an observation spans them, because some of the observed labour may have occurred outside the window. Put $d_k = b_k - b_{k-1} > 0$ and let x_{ijrk} denote the labour allocated to stream (i, j, r) in cell k. The following result shows that these cell totals preserve exactly the possible target values.

Proposition 4.2 (Exact cell reduction and attainable labour range)**.** *Under the preceding model, define the finite-dimensional feasible set $\mathcal{P}$ by*

$$x_{ijrk} \geq 0, \qquad \sum_{i,r} x_{ijrk} \leq d_k \qquad \text{for every } j, k, \tag{8}$$

$$v_\nu^- \leq \sum_{(i,j,r) \in G_\nu} \sum_{k:\, [b_{k-1}, b_k] \subseteq [\ell_\nu, u_\nu]} x_{ijrk} \leq v_\nu^+ \qquad \text{for every } \nu. \tag{9}$$

Fix real weights w_{ijrk} and define the target for a continuous-time allocation history by multiplying each cell integral by w_{ijrk} and summing. The attainable target values over histories satisfying the capacity and observation constraints equal $h(\mathcal{P})$, where $h(x) = \sum_{i,j,r,k} w_{ijrk} x_{ijrk}$. In particular, the denominator in (5) uses $w_{ijrk} = 1$ exactly when $i \in \mathcal{C}$ and $[b_{k-1}, b_k] \subseteq [s_i, c_i]$, and zero otherwise. If $\mathcal{P}$ is nonempty, its target range is the closed interval

$$[H_{\min}, H_{\max}] = \left[\min_{x \in \mathcal{P}} h(x),\ \max_{x \in \mathcal{P}} h(x) \right]. \tag{10}$$

Both endpoints and every value between them are attained, even if every $a_{ijr}(t)$ is restricted to $\{0, 1\}$, so that each person does at most one type of work on one task at a time. If $\mathcal{P}$ is empty, the observations and capacity assumptions are inconsistent.

Proof. Integrating any such history over each cell gives nonnegative cell labour totals. Integrating each person's allocation limit gives (8). Splitting each observation at the cell boundaries gives (9). The weighted target total is unchanged because its weights are constant on cells.

Conversely, fix $x \in \mathcal{P}$. For each person and cell, order the task–type-of-work pairs and divide the cell into consecutive blocks whose lengths are their respective x_{ijrk}. The capacity constraint ensures that these blocks fit inside the cell. Use half-open blocks to avoid overlap at their boundaries. During each block, set the corresponding allocation to one and all others to zero. Leave any remaining time idle. Omit zero-length blocks and give the final domain endpoint the same allocation values as the last block. These measurable allocations reproduce every cell labour total, so all observation bounds remain satisfied and the target value is unchanged. Each person works on at most one task and type of work at a time, with finitely many changes. Thus even these restricted allocation functions give exactly the same possible target values as the finite feasible set.

The constraints define a closed convex set. Each coordinate is between zero and d_k, so a nonempty $\mathcal{P}$ is compact and a linear functional attains its extrema. Convex combinations of minimising and maximising cell vectors attain every intermediate value. Reconstructing each such vector by consecutive blocks supplies an allocation with values in $\{0, 1\}$ for each intermediate total. □

This reduction makes the extreme target totals computable by linear programmes, each with $|\mathcal{I}||\mathcal{J}||\mathcal{R}|K$ variables, $|\mathcal{J}|K$ shared-capacity constraints and $2M$ observation inequalities, in addition to nonnegativity. A known period of unavailability can be included by adding its boundaries to the partition and setting that person's cell labour totals to zero within it.

The reconstruction realises fractional cell totals with one human activity at a time. It uses at most $|\mathcal{I}||\mathcal{R}| + 1$ constant blocks per person in each cell, including idle time. The reconstruction requires work to be allowed to start, stop and change type or task at arbitrary times within a cell. Minimum session lengths, restrictions on interruption, task dependencies, smoothness constraints and switching costs require a separate analysis.

The constructed schedule is one history of human activity consistent with the observed interval totals and weighted labour targets. AI workflows may keep executing concurrently while the person changes task. The equivalence concerns the labour quantities constrained by the model. Reproducing the actual order of task execution, acceptance times or output would require further information and constraints. Allocations satisfying these extra restrictions form a subset of the model's feasible allocations. Their labour totals therefore lie between the calculated minimum and maximum, although attainability for the restricted workflow requires a separate analysis.

Preserving the integrals while replacing fractional allocations by one activity at a time is a finite-partition instance of the Dvoretzky–Wald–Wolfowitz purification theorem [11, Theorem 4]. See also [17, Section 2]. The consecutive-block construction gives the required history directly and bounds its number of blocks.

4.3. **Checking the bounds and allowing observation error.** To verify a proposed maximum, give a feasible allocation that achieves it and derive an inequality from the observation and capacity constraints showing that every feasible target total is at most that value. To verify a proposed minimum, give a feasible allocation that achieves it and derive an inequality showing that every feasible target total is at least that value. The allocation establishes attainability, while the inequality bounds every feasible total. Equality between the bound and the attained total establishes optimality.

Collect the cell totals and target weights into vectors x and w, and write the capacity and observation inequalities as $Ax \le b$, with $x \ge 0$ kept separate. A lower observation bound is included by reversing its sign. Nonnegative multipliers λ and μ satisfying

$$A^{\mathsf{T}}\lambda \ge w, \qquad A^{\mathsf{T}}\mu \ge -w \quad \Longrightarrow \quad -\mu^{\mathsf{T}}b \le h(x) \le \lambda^{\mathsf{T}}b \qquad (x \in \mathcal{P}) \tag{11}$$

give bounds by taking weighted sums of the original inequalities, yielding $w^{\mathsf{T}}x \le \lambda^{\mathsf{T}}Ax \le \lambda^{\mathsf{T}}b$, and likewise for $-w$.

For a nonempty $\mathcal{P}$, linear-programming strong duality ensures that suitable multipliers give exactly $H_{\min}$ and $H_{\max}$ [6, Sections 5.2–5.3]. When the bound calculated from the multipliers equals the total achieved by a feasible allocation, together they prove that the total is the minimum or maximum. If the problem data, cell totals and multipliers are rational, these checks can use exact arithmetic without repeating the optimisation. A numerical solver's success flag alone does not supply this check.

With the partition and target fixed, widening observation limits changes only the right-hand sides of the inequalities. Increasing an upper observation bound or decreasing a lower one increases the corresponding entry of b. If these changes give $b + \varepsilon$, with $\varepsilon \geq 0$ and zero entries for unchanged constraints, the original multipliers still bound every newly permitted target total by

$$[-\mu^{\mathsf{T}}(b + \varepsilon),\ \lambda^{\mathsf{T}}(b + \varepsilon)]. \tag{12}$$

The same weighted sums of inequalities give these bounds, as in the sensitivity analysis of [6, Section 5.6]. Reoptimising may give a smaller interval. The observation process must still justify the error limits.

Sampled observation limits require a coverage argument. Suppose random lower and upper bounds contain all true observation totals simultaneously with probability at least $1 - \alpha$, where $0 < \alpha < 1$. If the true history satisfies the allocation constraints, its cell vector belongs to the resulting random feasible set on that event. Optimising then gives an interval containing the true target total with probability at least $1 - \alpha$. An empty feasible set counts as a failure to cover.

This guarantee concerns simultaneous coverage of all observation totals, so it requires an argument for the actual sampling and measurement process. Coverage of $1 - \alpha$ for each observation separately does not establish the same probability of covering them all. By the union bound, marginal error probabilities whose sum is at most α suffice. The resulting interval has a sampling-coverage guarantee, rather than the exact-data interpretation of (10).

A separate development in Lean 4, using mathlib [9, 22], checks that interval integrals equal sums of cell totals and that integrating each person's allocation limit gives the cell-capacity constraint. It constructs schedules of human activity in which each person does at most one activity at a time, preserving the observation totals and weighted labour targets. Concurrent AI execution remains part of the separate computing-resource account described in Section 3. The formal development also checks the bounds from supplied allocations and dual multipliers, and the implication from simultaneous coverage of observation totals to coverage of the target total. The existence of minimising and maximising allocations and dual multipliers, and the bound on the number of blocks in the reconstruction above, remain outside this formal supplement. The Rust solver is tested separately. The supplement does not prove that the Rust solver's source code implements the Lean definitions.

4.4. **From labour bounds to productivity.** Use the labour range corresponding to the denominator in (5), and let $Q = \sum_{i \in \mathcal{C}} Q_i(\tau, \Delta)$ be the total output score at follow-up, known exactly. With $Q \geq 0$ and $H_{\min} > 0$, dividing this output by the largest and smallest possible labour totals gives the identified productivity interval $[Q/H_{\max}, Q/H_{\min}]$.

When the output score is also uncertain, separate output and labour bounds leave open which pairs can occur together. Justified output-score bounds $0 \leq Q_- \leq Q \leq Q_+$ give an interval $[Q_-/H_{\max}, Q_+/H_{\min}]$ containing every feasible output-to-labour ratio. It is the exact range only when every ratio in it can be produced by an output total and labour total that are jointly permitted. It is sufficient for every pair in $[Q_-, Q_+] \times [H_{\min}, H_{\max}]$ to be permitted. A lower labour bound of zero cannot be used as a divisor. The labour optimisation itself holds the acceptance times and the recorded fractions of requirements still met in (4) fixed.

5. Estimating labour from sampled allocations

The preceding bounds describe every labour total permitted by given interval observations. Sampling raises the question of how selected person-intervals can support an estimate of total labour. The estimate varies with the selected intervals, so entering it as an exact observation can exclude the true allocation through sampling error. We therefore develop its sampling properties separately. The difference estimator below uses sampled labour to correct a telemetry prediction total. Under its sampling and measurement assumptions, the corrected estimate averages to the true labour total over repeated samples.

The sampling frame determines which work the estimate can represent. This complete list of person-intervals must cover all target work, including off-keyboard reasoning, review and coordination. Short diaries and task records can show how much of each selected interval a person spent on that work. Assess recording accuracy through independent checks of a subsample. Record the reviewers, evidence examined and resolution of each discrepancy. Report disagreement and investigate its causes. Nonresponse and inaccurate recall concern the records within the frame, while omitted work concerns the frame's coverage. Each requires its own investigation.

5.1. Predictions calculated before sampling.

Summing predictions for every person-interval gives a predicted labour total. Observations from a probability sample then estimate the difference between true labour and that prediction total. In each sampled interval, observed labour minus predicted labour is the residual. Dividing each residual by its inclusion probability and summing gives a correction that, under the conditions below, averages over repeated samples to the true total minus the prediction total. Denis and Haddou give this fixed-prediction estimator and its variance under the sampling design explicitly [10, Section 2.3, equations (2)–(4)]. Prediction-powered inference also identifies its estimator of a mean as a difference estimator and considers finite-population extensions [3, Section 1.4 and Appendix B of the accompanying preprint].

Partition each person's target observation period into non-overlapping intervals. Let $U = \{1, \dots, M\}$, $M \geq 1$, be the complete, fixed frame of these person-intervals. For interval m, let $d_m > 0$ be its length in hours and $a_m \in [0, 1]$ its average allocation to the target work. Its labour is $y_m = d_m a_m$ person-hours, and the population total is $H = \sum_U y_m$.

For every frame unit, retain a prediction f_m in person-hours. Convert a Standard Coding Hours value to predicted labour in person-hours before using it as f_m, so that the prediction and observed labour measure the same quantity. The conversion rule must identify the calculation used and its basis. The proof also requires each unit's prediction to remain the same whichever sample is drawn. A rule trained on an independent earlier population can satisfy this condition when all predictions are calculated and held fixed before selecting the correction sample.

Proposition 5.1 (Fixed-prediction difference estimator)**.** *For a probability sample $s \subseteq U$ with known first-order inclusion probabilities $\pi_m > 0$, set $r_m = y_m - f_m$ and*

$$\widehat{H}_{\text{diff}} = \sum_{m \in U} f_m + \sum_{m \in s} \frac{y_m - f_m}{\pi_m}. \tag{13}$$

If y_m and f_m are fixed under the design and every sampled labour value is observed accurately, then $\mathbb{E}_s[\widehat{H}_{\text{diff}}] = H$, where $\mathbb{E}_s$ averages over the sampling design.

Proof. Write $I_m = \mathbf{1}\{m \in s\}$. Linearity and $\mathbb{E}_s[I_m] = \pi_m$ give

$$\mathbb{E}_s[\widehat{H}_{\text{diff}}] = \sum_U f_m + \sum_U \frac{\mathbb{E}_s[I_m]}{\pi_m}(y_m - f_m) = \sum_U y_m.$$

□

The correction therefore preserves design-unbiasedness even when the fixed prediction total differs from H. Taking $f_m = 0$ gives the Horvitz–Thompson estimator, labelled HT in the equations [13],

$$\widehat{H}_{\mathrm{HT}} = \sum_{m \in s} \frac{d_m a_m}{\pi_m}. \tag{14}$$

The records must distinguish observed zeros from missing observations. For every frame interval, record whether it was selected or unselected. A selected interval then needs either its observed allocation, including zero, or a nonresponse record when no allocation value was recorded. These states distinguish a measured absence of target work from a gap in measurement. The proof concerns true allocation values, so unbiasedness in the presence of measurement error requires additional assumptions on that error.

5.2. **Precision depends on residuals and the sampling design.** The residuals and sampling design jointly determine precision. Let $\pi_{mk} = \mathbb{E}_s[I_m I_k]$ be the probability that both frame units are selected, including $\pi_{mm} = \pi_m$. The exact variance over repeated samples is

$$\mathrm{Var}_s(\widehat{H}_{\mathrm{diff}}) = \sum_{m,k \in U} \frac{\pi_{mk} - \pi_m \pi_k}{\pi_m \pi_k} r_m r_k. \tag{15}$$

Expanding the variance of $\sum_U I_m r_m / \pi_m$ gives this expression. Its covariance terms depend on how often pairs of units are selected together. Calculating the variance from (15) would also require residuals for unsampled units, which are generally unknown. Estimating it therefore requires the actual sampling design and satisfaction of the chosen variance estimator's conditions on joint inclusion probabilities.

For simple random sampling without replacement from $M \geq 2$ units, let the fixed sample size satisfy $1 \leq n \leq M$. Every size-n subset has the same probability and $\pi_m = n/M$. Write

$$S_r^2 = \frac{1}{M-1} \sum_U (r_m - \bar{r}_U)^2, \qquad S_y^2 = \frac{1}{M-1} \sum_U (y_m - \bar{y}_U)^2.$$

The inclusion covariances in (15) give

$$\mathrm{Var}_s(\widehat{H}_{\mathrm{diff}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_r^2, \qquad \mathrm{Var}_s(\widehat{H}_{\mathrm{HT}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_y^2. \tag{16}$$

For $2 \leq n \leq M$, an unbiased estimator of the design variance is

$$\widehat{V}_{\mathrm{diff}} = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) s_r^2, \qquad s_r^2 = \frac{1}{n-1} \sum_{m \in s} (r_m - \bar{r}_s)^2. \tag{17}$$

Indeed, $s_r^2 = [n(n-1)]^{-1} \sum_{m<k,\, m,k \in s} (r_m - r_k)^2$. Each pair has inclusion probability $n(n-1)/[M(M-1)]$, so $\mathbb{E}_s[s_r^2] = S_r^2$. A one-unit sample from a larger population cannot supply this variance estimate. Measuring every unit gives zero sampling variance, including in a single-unit population. Errors in the recorded allocations can still affect the labour estimate, so a census also needs measurement checks.

For $n < M$, both variances contain the same positive sampling factor, so the difference estimator improves precision over the Horvitz–Thompson estimator exactly when $S_r^2 < S_y^2$. The residuals must vary less across person-intervals than the labour values themselves. For example, consider $M = 4$, $n = 2$, and $y = (0, 2, 4, 6)$. The true total is 12, and the Horvitz–Thompson variance is $80/3$. Using $f = (1, 3, 3, 5)$ for every sample yields variance $16/3$, while using $f = (6, 4, 2, 0)$ yields $320/3$. The second predictor ranks intervals in reverse order of their true labour, amplifying residual variation and giving four times the Horvitz–Thompson variance. The difference estimator remains design-unbiased with either predictor.

Enumerating all six samples reproduces these variances and confirms that each estimator averages to the true total. It also verifies that (17) averages to each corresponding variance. These finite checks establish the calculations for the two predictors. A workplace study must determine whether its predictions derived from code changes give a residual correction with lower sampling variance than the labour estimate without predictions.

5.3. **Failure cases and implementation checks.** Fitting predictions to the correction sample can change the estimator's expectation. For $M = 2$, $n = 1$ and $y = (0, 2)$, suppose a fitted rule memorises the selected outcome and predicts zero for the other unit. Its observed residual is always zero, so the correction leaves the fitted total unchanged. The resulting difference estimate is 0 or 2, each with probability $1/2$, giving expectation 1 rather than $H = 2$. A method trained on the sampled units therefore needs a separate unbiasedness argument under the actual finite-population sampling design.

Clipping can also change the estimator's expectation. The estimate can be negative even though true labour is nonnegative. With $y = (0, 0)$, $f = (10, 0)$ and one uniformly selected unit, (13) is -10 or 10 with equal probability. Clipping at zero changes its expectation from 0 to 5. Reporting the signed estimate preserves the estimator in (13) while leaving an implausible result available for investigation.

Reporting a productivity ratio also needs a bias analysis, since dividing retained output by an unbiased labour estimate need not yield an unbiased productivity estimate.

The accompanying implementation reports estimates with their calculated sign, preserves observed zeros, and records nonresponse separately from unselected units. It rejects inputs with missing required information, selected intervals without allocation measurements, and inconsistent values. For a supplied general sampling design it reports point estimates. For simple random sampling it also reports variance and standard error under the conditions above.

These input checks establish whether supplied records satisfy the implementation's rules. Evidence from collection must establish that the frame covers the work, allocations are measured accurately and units were sampled by the method used to calculate their inclusion probabilities.

Exhaustive finite tests check the examples above, input validation, complete censuses and single-unit populations. A confidence interval requires a separate argument that its claimed coverage holds under the sampling design.

6. Validating the measurements

The mathematical results depend on observations that cover the target work and measure its person-hours accurately. A measurement pilot must test these premises before comparing AI tools. Define the included tasks and types of work, how hours are attributed to them and which person-intervals are available for sampling. Record the reference scoring rule and acceptance deadline together with the retained-output criteria before collection. These choices determine what the measurements describe.

The pilot must establish how reliably and at what effort those records can be collected. Include periods with concurrent AI workflows when checking attribution of shared oversight and attended waiting against the inclusion rules. Record disagreement over allocations and nonresponse, and track follow-up completeness and collection time. Report each by comparison group, type of work and explicitly defined task category. This evidence informs the sampling effort and precision that a later comparison can support.

Validation must also address the particular guarantee being used. Interval bounds require independently checked totals with justified limits on measurement error. For target windows chosen before observing outcomes, compare the sums of separately calculated lower and upper labour bounds with the joint minimum and maximum. This shows whether accounting jointly for each person's available time raises the minimum or lowers the maximum.

For sampled labour, the question is whether fixed predictions improve precision. Compare the variance over repeated samples with and without predictions calculated before sampling and kept unchanged whichever sample is drawn. Retain the inclusion probabilities and joint sampling-design information needed to calculate uncertainty. The following subsection explains how sampling error also affects the assessment of prediction accuracy.

6.1. **Comparing predictions with sampled labour.** Comparing a prediction with sampled labour combines prediction error with sampling error. Use labour recorded for teams or periods excluded when fitting the prediction rule, keeping predictions unchanged whichever validation sample is drawn. The following calculations separate prediction error against true labour from sampling variance.

Treat each task's labour H in a finite validation population as fixed. Let f be its prediction, calculated before validation sampling and unchanged whichever sample is drawn, and suppose $\mathbb{E}_s[\widehat{H}] = H$ with finite design variance. Then

$$\mathbb{E}_s[f - \widehat{H}] = f - H,$$
$$\mathbb{E}_s[(f - \widehat{H})^2] = (f - H)^2 + \mathrm{Var}_s(\widehat{H}).$$

Expanding the squared difference proves the second identity because the cross term has expectation zero when $\mathbb{E}_s[\widehat{H} - H] = 0$. The signed comparison therefore recovers prediction error on average, while the squared comparison also includes sampling variance. These identities hold for each task under the sampling design used to select its observed intervals. Aggregate uncertainty additionally depends on covariances between tasks' estimates.

For a second prediction g calculated before the same validation sample and unchanged whichever sample is drawn, subtracting the expected squared errors cancels the common sampling variance, giving

$$\mathbb{E}_s[(f - \widehat{H})^2 - (g - \widehat{H})^2] = (f - H)^2 - (g - H)^2.$$

Over repeated validation samples, the average difference between the two squared prediction errors equals their difference calculated against true labour. Estimating either predictor's squared error against true labour, or quantifying sampling uncertainty in their comparison, requires further sampling-design information.

The squared-error cancellation leaves open how sampling noise affects absolute error and interval coverage. To examine these measures, consider eight one-hour person-intervals with allocations $(1, 1, 1, 1, 0, 0, 0, 0)$. Sample one uniformly, with inclusion probability $1/8$. True labour is $H = 4$, whereas its Horvitz–Thompson estimate is 0 or 8, each with probability $1/2$. A perfect prediction $f = 4$ consequently has apparent absolute error 4 against the sampled estimate. The interval $[3.5, 4.5]$ always contains true labour but never contains the sampled estimate. Both apparent failures arise from comparing an accurate prediction or interval with a noisy labour estimate. The signed difference between prediction and sampled estimate remains unbiased for the prediction's error against true labour. Absolute error and interval coverage calculated against $\widehat{H}$ measure agreement with the sampled estimate, including its sampling error.

Prediction validation must therefore distinguish the prediction's error from uncertainty in the labour measurements. One way to obtain task-level reference measurements is to record allocations for every person-interval in each target window for a validation subset of tasks. Independently check those records, identifying the reviewers and evidence used to resolve discrepancies. State how checking remains independent of recording, and report remaining measurement error. Alternatively, use a measurement model under which the observations uniquely determine the chosen accuracy measure. Retain inclusion probabilities and joint sampling-design information for uncertainty calculations. Report systematic overstatement or understatement and the size of individual prediction errors by study group, type of work and explicitly defined task category. Correlation between group averages alone leaves systematic prediction bias and individual prediction errors unresolved.

The same validation must establish what the output scores represent, so prediction accuracy can be interpreted alongside the work delivered. Test whether task splitting, code-change squashing or changes to the task information supplied to the scoring rule alter the reference score. Report reviewer disagreement and follow-up completeness alongside prediction error. Recalculate comparisons between study groups using plausible amounts of missing labour and disputed retained-output scores. If plausible measurement errors reverse which group has the better outcome, the choice between AI tools and working practices depends on how that uncertainty is resolved or accepted.

6.2. **Conditions for comparing AI tools and working practices.** Accurate labour and output measurements support a comparison between groups. Attributing a difference to AI access also requires a causal design that defines which work its conclusion concerns. AI adoption can change which work is requested or undertaken, so comparing a fixed workload and studying an organisation's incoming work answer different questions. For a fixed-workload comparison, set the conditions for including tasks before allocating tasks, people or teams to comparison groups, and keep those conditions unchanged. State how work done before group allocation is counted, and retain records of postponed, rejected and abandoned requests. An organisation-wide adoption comparison needs a common calendar window and records of incoming requests and work in progress. Each design must identify its target population and explain how observations are weighted to represent it.

Group formation must account for work shared between teams. Reviewers, limited computing resources for agents and reusable components can transmit the effects of one team's AI access to another. The group boundaries and outcome analysis must therefore account for these routes of influence. Analyse outcomes by the original group allocation, recording actual tool use separately.

Whether teams, people or other units are allocated to groups determines the level at which causal uncertainty must be calculated. More commits within a team add records while leaving the number of independently allocated teams unchanged. The uncertainty calculation must also account for sampling labour and checking quality in subsets.

Retained-output comparisons require common criteria and a common follow-up procedure across groups. Two reviewers should check a subset independently so their disagreement can be measured. The check can itself trigger repair, making the timing of feedback and any loss of reviewer blinding—reviewers learning which group a task belongs to—part of the measurement record. Count resulting repair within the labour window used for the productivity denominator. Extending retained-output estimates from a checked subset to all tasks requires known selection probabilities and assumptions addressing any effect of the check on subsequent work. Report severe defects separately because a retained fraction cannot describe every security or reliability consequence.

Interpreting an apparent labour saving requires the corresponding acceptance and retained-output records. Complete follow-up through $\tau + \Delta$ as required by Section 3, distinguishing permanent abandonment from loss of observation. Incomplete follow-up requires explicit assumptions or bounds for missing outcomes. Report labour, acceptance and retained output together, since reduced labour may accompany more abandonment. Present retained output and positive total labour alongside their productivity ratio, accounting for uncertainty in both quantities and their dependence.

The decision criterion must then specify how much improvement is useful and how much quality loss is acceptable. Prespecify the improvement thresholds and the maximum acceptable reduction in quality (the noninferiority margin). Establishing that a loss lies within this margin requires evidence about its magnitude and uncertainty. Failure to detect a reduction alone leaves that requirement unresolved. The following experiment makes these criteria explicit for a fixed-workload comparison.

6.3. **A proposed causal experiment.** A fixed-workload experiment would compare teams given a recorded AI toolset and working practices with teams using the same ordinary development and review process without generative AI. Record each team's task requirements, acceptance criteria and reference scores before choosing groups, so its outcomes refer to work specified before allocation. Pair teams using repository domain, previous workload and experience, then use an independent fair coin within each pair to choose which team receives AI access [15]. The groups remain parallel throughout. The primary analysis follows that allocation, with actual tool use recorded separately.

The team boundaries also need to reflect how work is shared. Shared reviewers, developers, computing capacity or code can transmit effects between teams [14]. If these links require allocating several teams together, revise the estimator weights and planning before randomisation to preserve the equal-team target.

Require each team's initial reference scores to have a positive total. Use that total to normalise both outcomes, dividing total labour through task age $\tau + \Delta$ and retained output by the same fixed denominator. Include labour on failed and abandoned tasks, and assign an output score of zero to tasks that miss the acceptance deadline. The two primary causal quantities are the mean differences between groups, with each team weighted equally. Estimate them by subtracting the comparator team's outcome from the AI team's outcome within each pair and averaging the differences.

A benefit claim requires both a worthwhile labour reduction and a retained-output loss smaller than a maximum chosen before observing outcomes. Failure to detect a quality reduction alone leaves that maximum-loss requirement unresolved [21].

The provisional criteria require a saving exceeding one person-hour per reference unit and a retained-output loss below five percentage points. For uncertainty and measurement bias, the protocol uses approximate one-sided 97.5% Student-t bounds from independent pair differences, expanded by 0.2 person-hours per reference unit for labour and 0.01 for the retained fraction. Both expanded bounds must meet their respective criteria. These measurement-bias allowances require empirical justification, and the practical thresholds require substantive agreement before a principal study. Any unresolved team labour or output outcome prevents a benefit claim under this plan.

The accompanying protocol specifies a separate six-pair measurement pilot and provisional $\tau = \Delta = 336$-hour clocks (14 calendar days each). Twenty thousand simulated experiments per design and scenario compare 48 and 96 teams under hypothetical outcome distributions, measurement bias and missing team outcomes, including cases at each benefit boundary [18]. Neither team count reaches the required 90% probability of meeting both criteria across the required useful-effect scenarios.

Before choosing a principal sample size, the pilot must establish defensible ranges for measurement error, variability and completeness of the team outcome records. Its observed treatment effect must not determine that choice [1]. The experiment remains specified and unrun. These hypothetical planning calculations report no causal workplace result.

7. Scope and next observations

Different distributions of human activity can produce identical commit distributions. Interval totals or justified bounds, together with each person's available time, restrict how much work could have occurred during a specified period. Splitting time at the boundaries of every observation interval and target window gives a finite optimisation problem with exactly the same possible labour totals. This joint formulation enforces the shared capacity that links a person's tasks. Adding separately calculated interval bounds can include totals inconsistent with that capacity. The summed lower and upper bounds are exact only if work allocations achieving the respective component bounds can occur together.

Probability sampling permits design-unbiased estimation when the frame covers the target work, inclusion probabilities are known and positive, and sampled allocations are measured

accurately. A fixed telemetry predictor improves precision only when estimating its total residual has lower sampling variance than estimating labour without predictions. Applying these guarantees requires evidence that the records measure the intended work. Diaries and task records can supply the observations, but whether they capture reasoning and coordination and accurately record how people divide their time requires empirical checking.

Establishing a causal benefit from AI adoption requires a comparison of AI tools and working practices. An organisation-wide adoption study would include changes to incoming requirements, whereas the proposed experiment concerns a fixed workload. The next empirical step towards that experiment is the measurement pilot specified in Section 6.3. It must test whether task records capture person-hours across the included types of work and support independent checks of retained output through follow-up. Disagreement, missing records and collection effort will inform the measurement procedure and size of the proposed fixed-workload experiment.

Appendix A. A shortest-path solution for one allocation stream

For one person's work of one type on one task, let $0 \le a \le 1$ be the allocation function. The observations of Section 4 bound labour within each observed interval. Expressing that labour as the difference between cumulative totals lets us encode the observations as graph edges. Using the sorted time boundaries and cell lengths defined there, set $z_k = \int_{b_0}^{b_k} a(t)\,dt$. Represent an inequality $z_q - z_p \le w$ by a directed edge $p \to q$ of weight w.

The graph combines capacity limits with the observed interval totals. Adjacent time boundaries receive edges $k-1 \to k$ of weight d_k and $k \to k-1$ of weight zero, encoding the maximum available time and nonnegativity. Observation bounds on $[b_p, b_q]$ add edges $p \to q$ of weight v^+ and $q \to p$ of weight $-v^-$. An exact observed labour total v therefore gives edge weights v and $-v$. This is the standard difference-constraints representation [16, Slides 6–11]. Path weights then bound differences in cumulative labour.

Proposition A.1 (Attainable cumulative bounds from graph distances)**.** *The observations for this single allocation function admit an allocation if and only if this graph has no directed cycle of negative total weight. When such an allocation exists, let $D(p,q)$ be the shortest-path weight from p to q. For any endpoint pair $p \le q$, the identified range of $\int_{b_p}^{b_q} a(t)\,dt$ is*

$$[-D(q,p),\ D(p,q)]. \tag{18}$$

Proof. Summing edge inequalities around a cycle gives 0 at the left, so a negative cycle is inconsistent. Conversely, adjacent edges make every vertex reachable from every other. In the absence of negative cycles, all shortest-path weights are finite and $D(r,r) = 0$. For any starting vertex r, the values $z_k = D(r,k)$ satisfy each edge inequality because $D(r,v) \le D(r,u) + w$ for an edge $u \to v$ of weight w. Subtracting z_0 makes initial cumulative labour zero without changing differences. The inequalities for adjacent endpoints give cell labour totals in $[0, d_k]$. Allocations constant within each cell can produce these totals and satisfy the observations.

Summing inequalities along paths yields $z_q - z_p \le D(p,q)$ and $z_q - z_p \ge -D(q,p)$. Using p as the starting vertex gives an allocation achieving the upper bound. Using q gives an allocation achieving the lower bound. Convex combinations of these allocations give every labour total between those bounds. □

The reference checker implements these bounds with exact rational arithmetic and an all-pairs shortest-path calculation. Given indexed, sorted time boundaries, graph construction and this calculation use $O((K+1)^3 + M)$ arithmetic operations and $O((K+1)^2 + M)$ storage. Bit costs depend on the encoded rational inputs. The checker compares the bounds and the allocations attaining them against an independent enumeration of finite allocations, including overlapping interval observations and inconsistent inputs.

Several task streams sharing a person's capacity require the joint formulation in Proposition 4.2, because the scalar graph represents only one allocation. The constructed examples check the calculation for that single-stream model. Applying it to workplace observations also requires empirical validation of the observation process.

References

1. Casper Albers and Daniël Lakens, *When power analyses based on pilot data are biased: Inaccurate effect size estimators and follow-up bias*, Journal of Experimental Social Psychology **74** (2018), 187–195, https://doi.org/10.1016/j.jesp.2017.09.004.
2. Elizabeth S. Allman, Catherine Matias, and John A. Rhodes, *Identifiability of parameters in latent structure models with many observed variables*, The Annals of Statistics **37** (2009), no. 6A, 3099–3132, Section 6.1, Theorem 6. https://jarhodesuaf.github.io/papers/Latent.pdf.
3. Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic, *Prediction-powered inference*, Science **382** (2023), no. 6671, 669–674, Author manuscript: arXiv:2301.09633v4. https://doi.org/10.1126/science.adi6000.
4. Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein, *Measuring the impact of early-2025 AI on experienced open-source developer productivity*, 2025, arXiv:2507.09089v2. https://arxiv.org/abs/2507.09089v2.
5. Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud, *We are changing our developer productivity experiment design*, METR research update, February 2026, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/.
6. Stephen Boyd and Lieven Vandenberghe, *Convex optimization*, Cambridge University Press, 2004, Sections 5.2–5.3 and 5.6. https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
7. Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, June 2025, Author manuscript; Section 4.1 and Table 3. https://www.mertdemirer.com/Papers/Demirer_AI_productivity.pdf.
8. ———, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, Management Science (2026), Published online 27 February 2026. https://doi.org/10.1287/mnsc.2025.00535.
9. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction—CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, https://lean-lang.org/papers/lean4.pdf, pp. 625–635.
10. Nicholas Denis and Mohammed Haddou, *Prediction-powered estimation: Unbiased model-assisted estimation*, Journal of Official Statistics **42** (2026), no. 3, 371–394, https://doi.org/10.1177/0282423X261451320.
11. A. Dvoretzky, A. Wald, and J. Wolfowitz, *Relations among certain ranges of vector measures*, Pacific Journal of Mathematics **1** (1951), no. 1, 59–74, Theorem 4. https://msp.org/pjm/1951/1-1/pjm-v1-n1-p06-s.pdf.
12. Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler, *The SPACE of developer productivity: There's more to it than you think*, ACM Queue **19** (2021), no. 1, 20–48, https://doi.org/10.1145/3454122.3454124.
13. D. G. Horvitz and D. J. Thompson, *A generalization of sampling without replacement from a finite universe*, Journal of the American Statistical Association **47** (1952), no. 260, 663–685, https://doi.org/10.1080/01621459.1952.10483446.
14. Michael G. Hudgens and M. Elizabeth Halloran, *Toward causal inference with interference*, Journal of the American Statistical Association **103** (2008), no. 482, 832–842, https://doi.org/10.1198/016214508000000292.

15. Kosuke Imai, Gary King, and Clayton Nall, *The essential role of pair matching in cluster-randomized experiments, with application to the mexican universal health insurance evaluation*, Statistical Science **24** (2009), no. 1, 29–53, https://doi.org/10.1214/08-STS274.
16. Piotr Indyk, *Lecture 17: Shortest paths IV: Applications*, 6.006 Introduction to Algorithms, Massachusetts Institute of Technology, 2011, Lecture slides, 7 April; difference constraints on slides 6–11. https://courses.csail.mit.edu/6.006/spring11/lectures/lec17.pdf.
17. M. Ali Khan, Kali P. Rath, and Yeneng Sun, *The Dvoretzky–Wald–Wolfowitz theorem and purification in atomless finite-action games*, International Journal of Game Theory **34** (2006), no. 1, 91–104, Section 2. https://doi.org/10.1007/s00182-005-0004-3.
18. Tim P. Morris, Ian R. White, and Michael J. Crowther, *Using simulation studies to evaluate statistical methods*, Statistics in Medicine **38** (2019), no. 11, 2074–2102, https://doi.org/10.1002/sim.8086.
19. Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz, *Reading between the lines: Modeling user behavior and costs in AI-assisted programming*, Proceedings of the CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, 2024, Author manuscript: arXiv:2210.14306v5. https://arxiv.org/abs/2210.14306v5, pp. 142:1–142:16.
20. Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer, *The impact of AI on developer productivity: Evidence from GitHub Copilot*, 2023, arXiv:2302.06590v1. https://arxiv.org/abs/2302.06590v1.
21. Gilda Piaggio, Diana R. Elbourne, Stuart J. Pocock, Stephen J. W. Evans, and Douglas G. Altman, *Reporting of noninferiority and equivalence randomized trials: Extension of the CONSORT 2010 statement*, JAMA **308** (2012), no. 24, 2594–2604, https://doi.org/10.1001/jama.2012.87802.
22. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, Association for Computing Machinery, 2020, https://leanprover-community.github.io/papers/mathlib-paper.pdf, pp. 367–381.
23. Alexander Torgovitsky, *Partial identification by extending subdistributions*, Quantitative Economics **10** (2019), no. 1, 105–144, https://doi.org/10.3982/QE634.
24. Ian Wright and Albert Ziegler, *The standard coder: a machine learning approach to measuring the effort required to produce source code change*, 2019, arXiv:1903.02436v1. https://arxiv.org/abs/1903.02436v1.